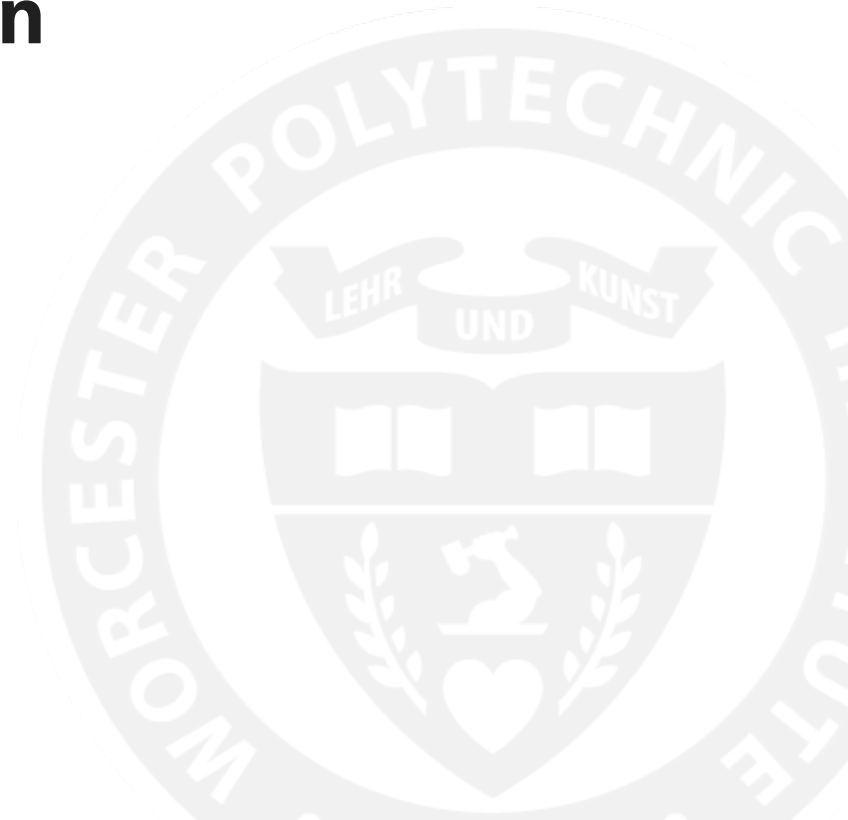




WPI

Multilayer perceptron



Multi-layer perceptron (MLP)

People gradually lost interests for neural networks due to the limit ability of perceptron. Things changed in 80s. The invention of Back Propagation algorithm (BP algorithm) make it possible to build neural network with multiple layers. *train weights*

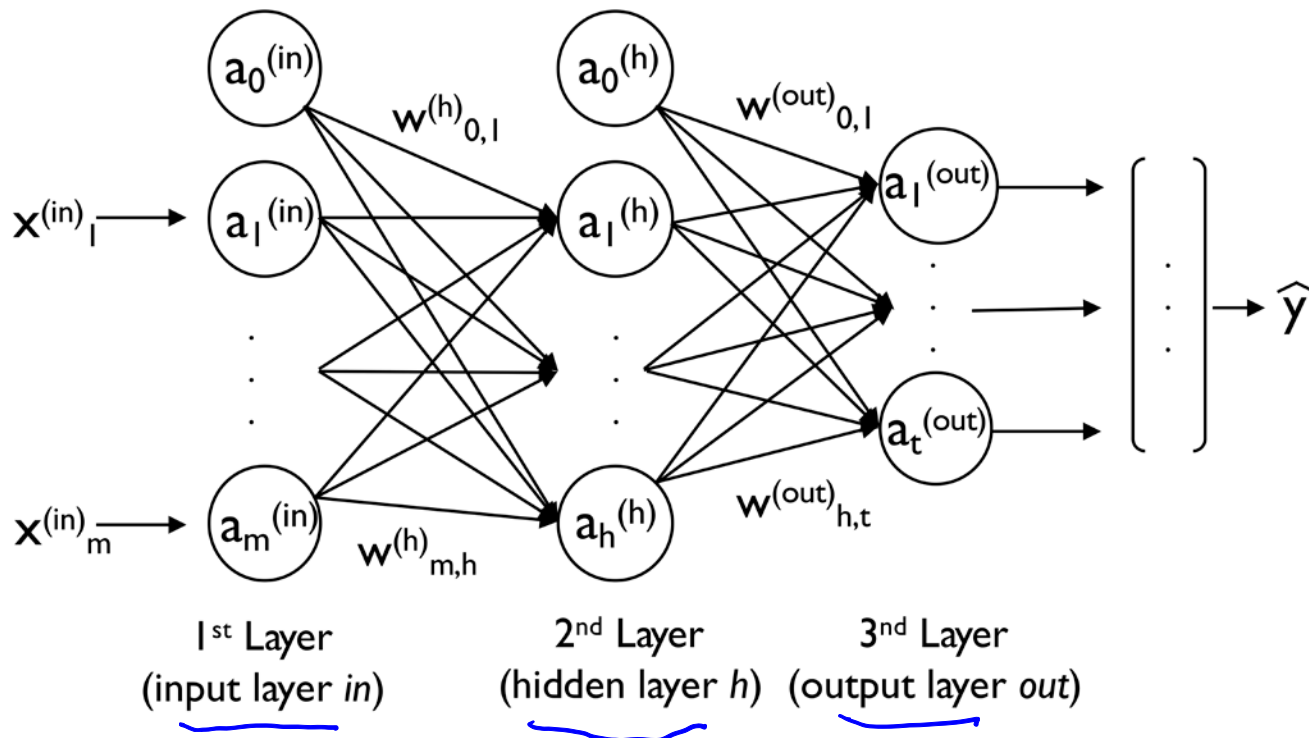
BP → neural network

Multi-layer perceptron can make non-linear decision boundary which is so common for real world problems. So, what is BP algorithm? How can it help build MLP? Why MLP is powerful?

Multi-layer perceptron (MLP)

What is Multi-layer perceptron?

Firstly, let's build a multi-layer neural network. This is a neural network with one hidden layer.



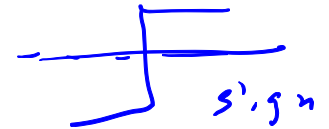
Multi-layer perceptron (MLP)

What is Multi-layer perceptron?

In single layer perceptron, there is just one neuron in output layer. Here, we expand it to multiple neurons.

Input:

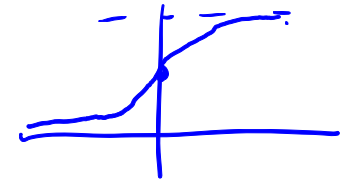
$$X^{in} = (x_1^{in}, x_2^{in}, x_3^{in}, \dots, x_m^{in})$$



Weights:

$$W^h = \begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,h} \\ w_{1,0} & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ w_{m,0} & \dots & \dots & \dots & w_{m,h} \end{bmatrix}$$

$$f(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid (logistic fn.)}$$



In hidden layer:

$$\underline{Z} = \underline{W}^T X \quad X_h^{out} = \underline{\phi(Z)}$$

$\phi()$ activation function

Multi-layer perceptron (MLP)

What is Multi-layer perceptron?

The weights for single neuron is a vector with the same size of input vector and the output is a single value.

The weights for multiple neurons is a matrix and the output is a vector with the same size of neurons.

$\phi(Z)$ is a non-linear activation function, we need it to help us draw non-linear decision boundary. It can be sigmoid function, or others.

The output layer:

$$Z_{out} = W_{out}^T X^{in}$$

$$X^{out} = \phi(Z_{out})$$

Multi-layer perceptron (MLP)

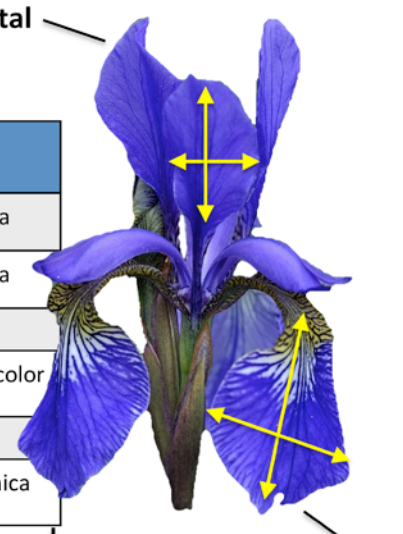
How to model multiple class classification?

So, until now, we have an output vector, one weights matrix for output layer and one weights matrix for hidden layer.

The next question is, how can we model multiple class classification with neural networks?

Do you still remember Iris dataset?

There are three classes here: Iris setosa=0,
Iris virginica =1,
Iris versicolor=2.



Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Features
(attributes, measurements, dimensions)

Class labels
(targets)

Petal

Sepal

Multi-layer perceptron (MLP)

How to model multiple class classification?

We use one-hot to transform them into vector:

Iris setosa = (1,0,0),

Iris virginica = (0,1,0),

Iris versicolor = (0,0,1).

This means we need to put 3 neurons on the output layer.

Then, we need to update weights for a goal. So, what is the goal?
What should be the cost function for multiple class classification problem?

We use logistic function as the activation function for 2 class classification problem. For multiple classes, we use **softmax function**:

$$\phi(Z)_i = \frac{e^{Z_i}}{\sum_{i=1}^3 e^{Z_i}}$$

output

layer output

Multi-layer perceptron (MLP)

How to model multiple class classification?

The output of **softmax function** is the normalized probability for a given sample belongs to a class. **It's still probability!**

Then, we can use **cross entropy** lost again:

One sample for 2-class :

$$\text{Cross-entropy} = -(\underbrace{y \log(p)} + \underbrace{(1 - y) \log(1 - p)})$$

One sample for n-class :

$$\text{Cross-entropy} = - \sum_{i=1}^n y_i \log(p_i)$$

$$\text{Total lost for m samples: } - \frac{\sum_m \sum_{i=1}^n y_i \log(p_i)}{m}$$

weight

BP

MSE

lost
function

$$\sum \frac{(y - \hat{y})^2}{m}$$

Multi-layer perceptron (MLP)

How to model multiple class classification?

m - samples
 n - classes

The other popular choice is the **mean square error (MSE)** lost:

$$\frac{1}{m} \sum_m \frac{\sum_{i=1}^n (\dot{y}_i - p_i)^2}{n}$$

regression problem

BP update weight

MSE keeps the same form for 2 or n classes, and it's always easy to be optimized. You can freely choose logistic or softmax function for output layer, since the MSE also can be used when the output is not the probability. MSE is widely used in the whole machine learning field.

How to update weights?

Now, we have the lost function. We can still use SGD to optimize

the model. The only different is how to calculate $\frac{\partial f}{\partial w_j}$ for hidden layer.

Multi-layer perceptron (MLP)

How to update weights?

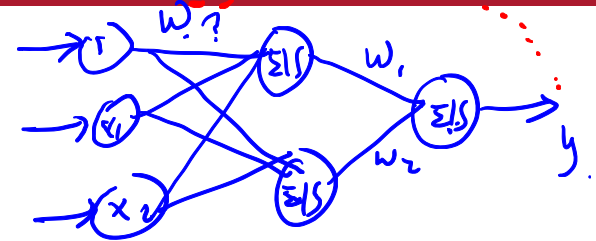
Here is the intuition of BP algorithm:

You may recall the concept of the chain rule from your introductory calculus classes. The chain rule is an approach to compute the derivative of a complex, nested function, such as $f(g(x))$, as follows:

$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Similarly, we can use the chain rule for an arbitrarily long function composition. For example, let's assume that we have five different functions, $f(x)$, $g(x)$, $h(x)$, $u(x)$, and $v(x)$, and let F be the function composition: $F(x) = f(g(h(u(v(x)))))$. Applying the chain rule, we can compute the derivative of this function as follows:

$$\frac{dF}{dx} = \frac{d}{dx} F(x) = \frac{d}{dx} f(g(h(u(v(x))))) = \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

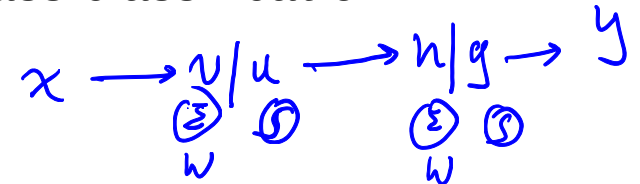


Multi-layer perceptron (MLP)

How to update weights?

Let's look at a 3 layers perceptron for a n-class classification.

For a input x , $v(x) = W_h^T x$



Sigmoid

$$u(v) = \frac{1}{1 + e^{-v}}$$

$$h(u) = W_{out}^T u$$

Sigmoid

$$g(h) = \frac{1}{1 + e^{-h}}$$

$$f(g) = -\frac{1}{n} \sum_{i=1}^n (y_i - g_i)^2 \quad \text{MSE}$$

Activation function,
non-linear transform.

Since y is the given label for input x , then we link everything together with the BP algorithm!

Multi-layer perceptron (MLP)

How to update weights?

Then, we can see, the training of multiple layer neural networks consists two parts, the feedforward pass which can give the inference and the backward pass which can update weights.

$$\phi'(z) = \frac{\partial}{\partial z} \left(\frac{1}{1+e^{-z}} \right)$$

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$

$$= \frac{1+e^{-z}}{(1+e^{-z})^2} - \left(\frac{1}{1+e^{-z}} \right)^2$$

$$= \frac{1}{1+e^{-z}} - \left(\frac{1}{1+e^{-z}} \right)^2$$

$$= \phi(z) - (\phi(z))^2$$

$$= \phi(z)(1 - \phi(z))$$

$$= a(1-a)$$

Let's try a simple 2-class example!

Before that, I will give you some tips for derivative.

$$\frac{df(g)}{dg} = \frac{2}{n}(y-g)g'(h)$$

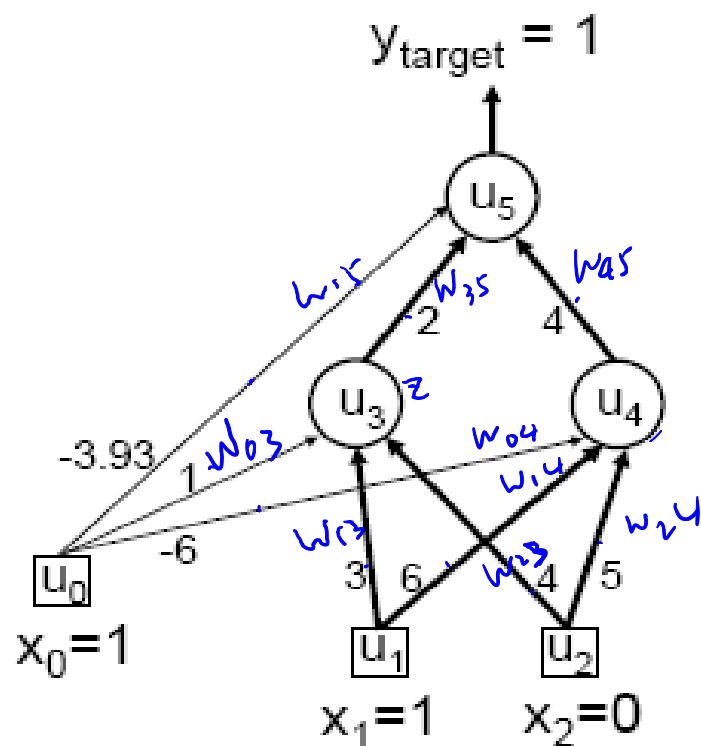
$$= (y-g)g(1-g)$$

$$\phi'(z) = \phi(z)(1-\phi(z))$$

$$y' = y(1-y)$$

Multi-layer perceptron (MLP)

Example: the model setting



$$u_3 := \phi(u_0 * 1 + u_1 * 3 + u_2 * 6)$$

$$u_4 := \phi(u_0 * (-6) + u_1 * 6 + u_2 * 5)$$

$$u_5 := \phi(u_3 * 2 + u_4 * 4)$$

Current state:

- Weights on arrows e.g. $w_{13} = 3$, $w_{35} = 2$, $w_{24} = 5$
- Bias weights, e.g. bias for unit 4 (u_4) is $w_{04} = -6$

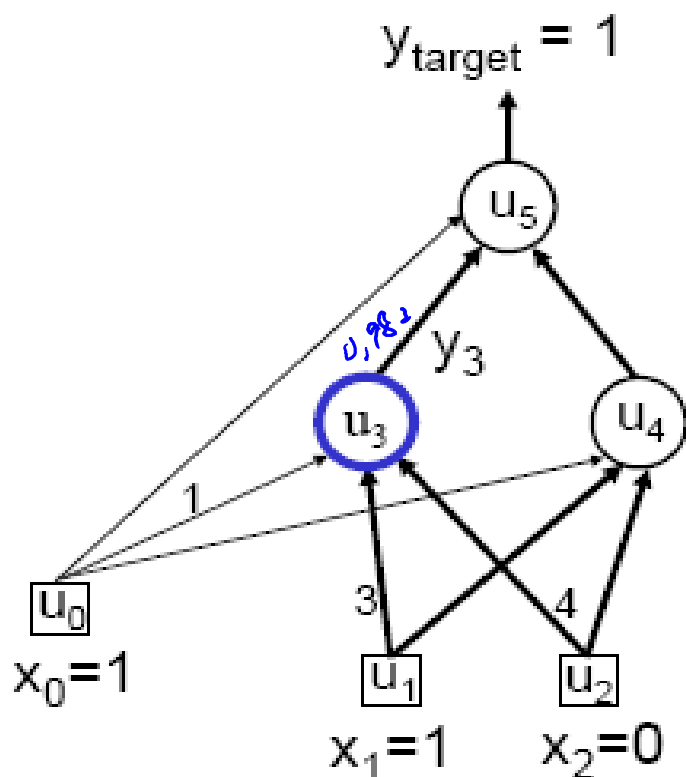
Training example (e.g. for logical OR problem):

- Input pattern is $x_1 = 1$, $x_2 = 0$
- Target output is $y_{\text{target}} = 1$

x_1	x_2	y	n

Multi-layer perceptron (MLP)

Example: forward pass



Output for any neuron/unit j can be calculated from:

$$a_j = \sum_i w_{ij} x_i$$

$$y_j = f(a_j) = \frac{1}{1 + e^{-a_j}}$$

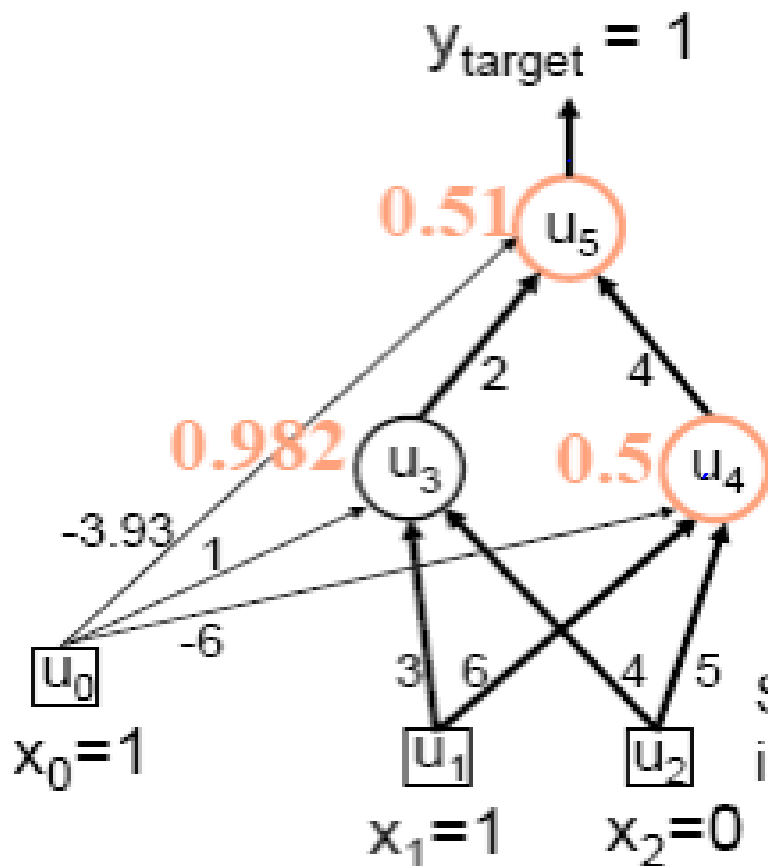
e.g Calculating output for Neuron/unit 3 in hidden layer:

$$a_3 = 1*1 + 3*1 + 4*0 = 4$$

$$y_3 = f(4) = \frac{1}{1 + e^{-4}} = 0.982$$

Multi-layer perceptron (MLP)

Example: forward pass



Unit	activation	output
	a_j	y_j
u_3	4.00	0.982
u_4	0.00	0.500
u_5	0.04	0.510

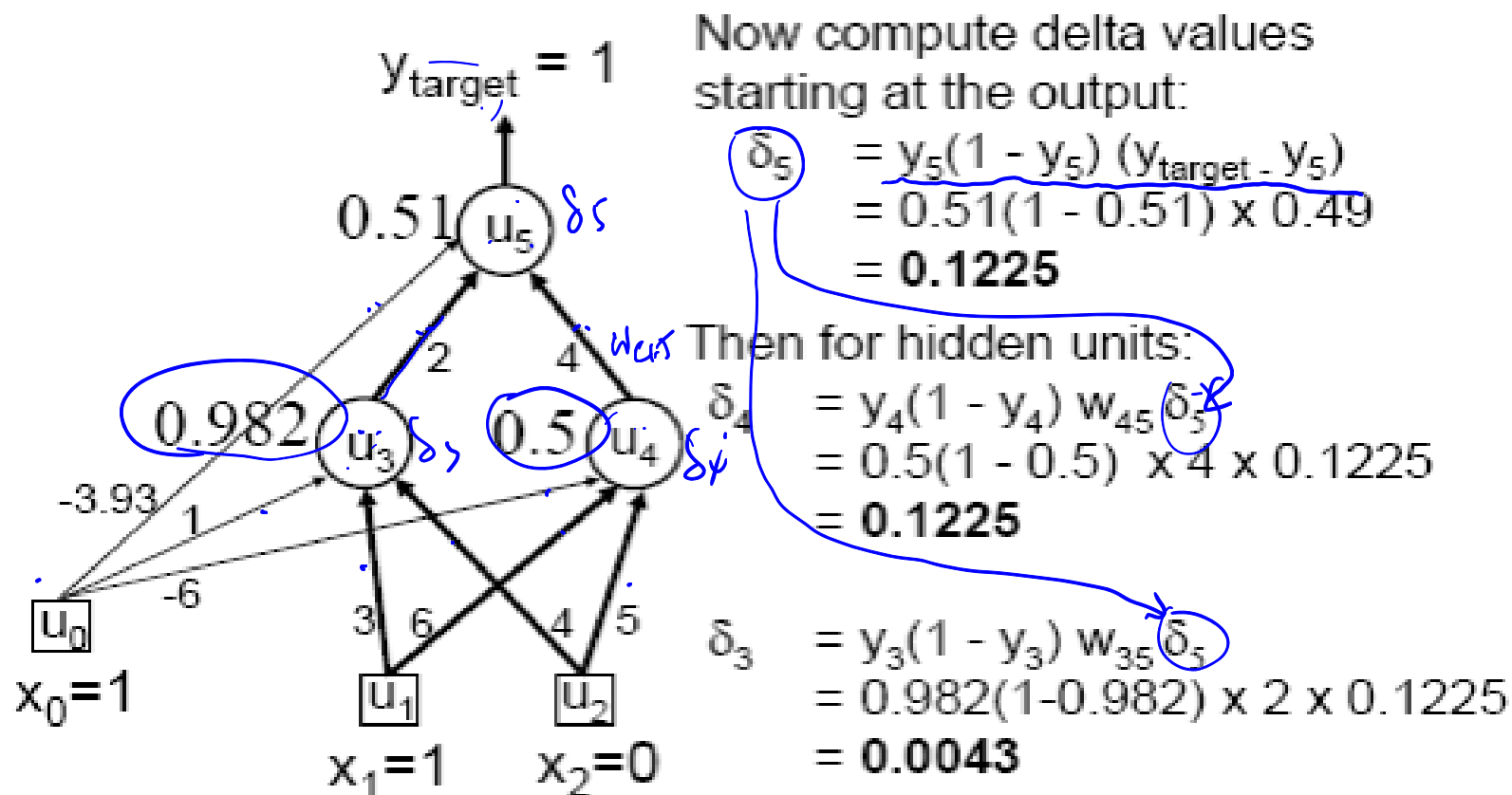
(network output)

So the error for this training example is: $(y_{\text{target}} - y_5) = (1 - 0.510) = 0.490$

update weights

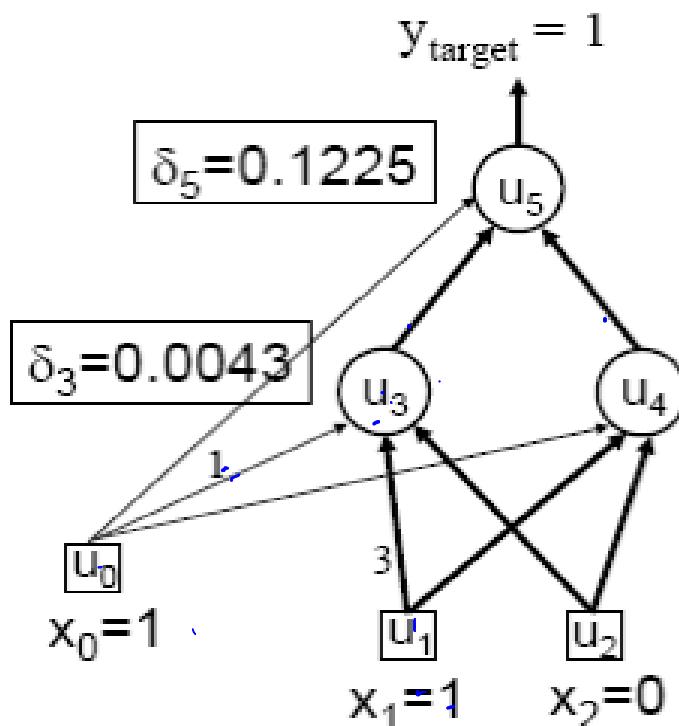
Multi-layer perceptron (MLP)

Example: backward pass



Multi-layer perceptron (MLP)

Example: update



- ◆ Set learning rate $\eta = 0.1$
Change weights by:

$$\Delta w_{ij} = \eta \delta_j x_i$$

- ◆ e.g. bias weight on u_3 :

$$\begin{aligned} \Delta w_{03} &= \eta \delta_3 x_0 \\ &= 0.1 * 0.0043 * 1 \\ &= 0.0004 \end{aligned}$$

So, new w_{03} $\boxtimes$

$$\begin{aligned} &w_{03}(\text{old}) + \Delta w_{03} \\ &= 1 + 0.0004 = 1.0004 \end{aligned}$$

- ◆ and likewise: $\eta \delta_j /$

$$\begin{aligned} w_{13} &\boxtimes 3 + 0.0004 \\ &= 3.0004 \end{aligned}$$

Multi-layer perceptron (MLP)

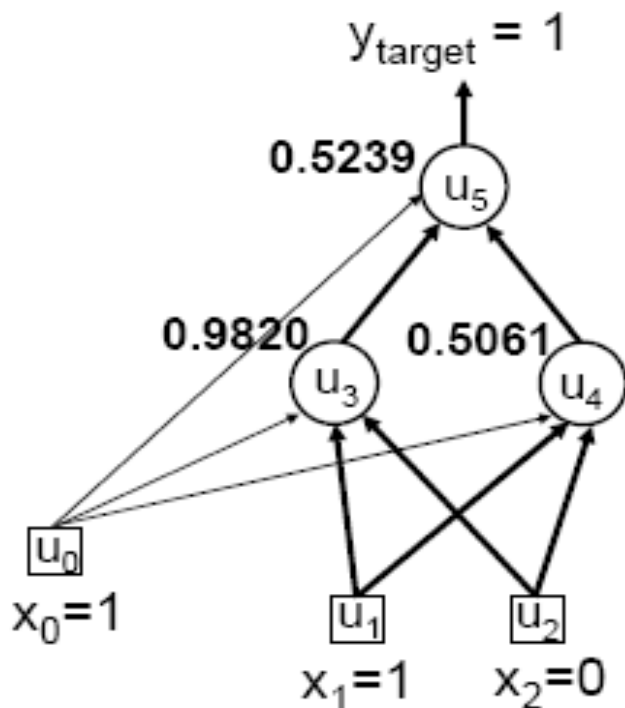
Example: update

Apply for all the weights:

i	j	w_{ij}	δ_j	y_i	Updated $w_{ij}^{new} = w_{ij}^{old} + \Delta w_{ij}$
0	3	1	0.0043	1.0 x_0	1.0004 w_{03}
1	3	3	0.0043	1.0 x_1	3.0004
2	3	4	0.0043	0.0 x_2	4.0000
0	4	-6	0.1225	1.0 x_0	-5.9878
1	4	6	0.1225	1.0 x_1	6.0123
2	4	5	0.1225	0.0 x_2	5.0000
0	5	-3.92	0.1225	1.0 x_0	-3.9078
3	5	2	0.1225	0.9820 u_3	2.0120
4	5	4	0.1225	0.5 u_4	4.0061 w_{45}

Multi-layer perceptron (MLP)

Example: verification



On next forward pass:

The new activations are:

$$y_3 = f(4.0008) = 0.9820$$

$$y_4 = f(0.0245) = 0.5061$$

$$y_5 = f(0.0955) = \mathbf{0.5239}$$

Thus the new error

$$(y_{\text{target}} - y_5) = (1 - 0.5239) = \mathbf{0.476}$$

has been reduced by 0.014
(from **0.490** to **0.476**)

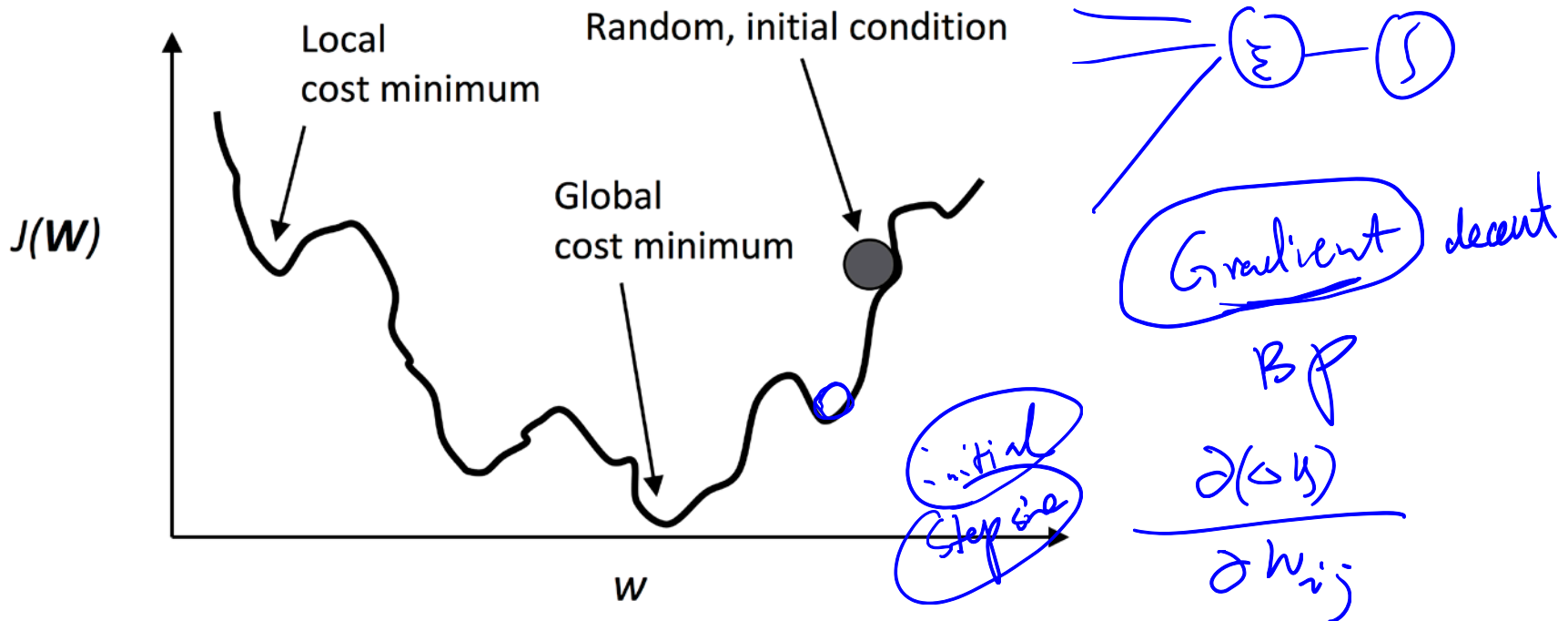
Ref: "Neural Network Learning & Expert Systems" by Stephen Gallant

update again
BP

Multi-layer perceptron (MLP)

The limitation of neural networks?

1. It works like black box.
2. The optimization is hard, it's easy to stuck in local minimum, since the function is complicated.





WPI

Thanks!

